

Exploratory Data Analysis (MySQL version)

Understanding the dataset to explore how the data is present in the database and if there is a need of creating some aggregated tables that can help with:

- Vendor selection for profitability
- Product Pricing Optimization

```
import mysql.connector
import pandas as pd

# creating database connection
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='your_password',
    database='inventory'
)

# checking tables present in the database
tables = pd.read_sql_query("SHOW TABLES", conn)
tables
```

	Tables_in_inventory
0	purchases
1	purchase_prices
2	vendor_invoice
3	begin_inventory
4	end_inventory
5	sales
6	vendor_sales_summary

```
# exploring what type of data available in the tables
for table in tables.columns[0]:
    print('-'*50,f'{table}','-'*50)
    print('Count of records:',pd.read_sql_query(f"SELECT COUNT(*) AS count FROM {table}",conn)['count'].values[0])
    display(pd.read_sql_query(f"SELECT * FROM {table} LIMIT 5",conn))
```

----- purchases -----
Count of records: 2372474

	InventoryId	Store	Brand	Description	Size \
0	69_MOUNTMEND_8412	69	8412	Tequila Ocho Plata Fresno	750mL
1	30_CULCHETH_5255	30	5255	TGI Fridays Ultimte Mudslide	1.75L
2	34_PITMERDEN_5215	34	5215	TGI Fridays Long Island Iced	1.75L
3	1_HARDERSFIELD_5255	1	5255	TGI Fridays Ultimte Mudslide	1.75L
4	76_DONCASTER_2034	76	2034	Glendalough Double Barrel	750mL

	VendorNumber	VendorName	PONumber	PODate \
0	105	ALTAMAR BRANDS LLC	8124	2023-12-21
1	4466	AMERICAN VINTAGE BEVERAGE	8137	2023-12-22
2	4466	AMERICAN VINTAGE BEVERAGE	8137	2023-12-22
3	4466	AMERICAN VINTAGE BEVERAGE	8137	2023-12-22
4	388	ATLANTIC IMPORTING COMPANY	8169	2023-12-24

	ReceivingDate	InvoiceDate	PayDate	PurchasePrice	Quantity	Dollars \
0	2024-01-02	2024-01-04	2024-02-16	35.71	6	214.26
1	2024-01-01	2024-01-07	2024-02-21	9.35	4	37.40
2	2024-01-02	2024-01-07	2024-02-21	9.41	5	47.05
3	2024-01-01	2024-01-07	2024-02-21	9.35	6	56.10
4	2024-01-02	2024-01-09	2024-02-16	21.32	5	106.60

	Classification
0	1
1	1
2	1

3 1
4 1

----- purchase_prices -----
Count of records: 12261

	Brand	Description	Price	Size	Volume	Classification \
0	58	Gekkeikan Black & Gold Sake	12.99	750mL	750	1
1	62	Herradura Silver Tequila	36.99	750mL	750	1
2	63	Herradura Reposado Tequila	38.99	750mL	750	1
3	72	No. 3 London Dry Gin	34.99	750mL	750	1
4	75	Three Olives Tomato Vodka	14.99	750mL	750	1

	PurchasePrice	VendorNumber	VendorName
0	9.28	8320	SHAW ROSS INT L IMP LTD
1	28.67	1128	BROWN-FORMAN CORP
2	30.46	1128	BROWN-FORMAN CORP
3	26.11	9165	ULTRA BEVERAGE COMPANY LLP
4	10.94	7245	PROXIMO SPIRITS INC.

----- vendor_invoice -----
Count of records: 5543

	VendorNumber	VendorName	InvoiceDate	PONumber	\
0	105	ALTAMAR BRANDS LLC	2024-01-04	8124	
1	4466	AMERICAN VINTAGE BEVERAGE	2024-01-07	8137	
2	388	ATLANTIC IMPORTING COMPANY	2024-01-09	8169	
3	480	BACARDI USA INC	2024-01-12	8106	
4	516	BANFI PRODUCTS CORP	2024-01-07	8170	

	PONumber	PayDate	Quantity	Dollars	Freight	Approval
0	2023-12-21	2024-02-16	6	214.26	3.47	None
1	2023-12-22	2024-02-21	15	140.55	8.57	None
2	2023-12-24	2024-02-16	5	106.60	4.61	None
3	2023-12-20	2024-02-05	10100	137483.78	2935.20	None
4	2023-12-24	2024-02-12	1935	15527.25	429.20	None

----- begin_inventory -----
Count of records: 206529

	InventoryId	Store	City	Brand	Description \
0	1_HARDERSFIELD_58	1	HARDERSFIELD	58	Gekkeikan Black & Gold Sake
1	1_HARDERSFIELD_60	1	HARDERSFIELD	60	Canadian Club 1858 VAP
2	1_HARDERSFIELD_62	1	HARDERSFIELD	62	Herradura Silver Tequila
3	1_HARDERSFIELD_63	1	HARDERSFIELD	63	Herradura Reposado Tequila
4	1_HARDERSFIELD_72	1	HARDERSFIELD	72	No. 3 London Dry Gin

	Size	onHand	Price	startDate
0	750mL	8	12.99	2024-01-01
1	750mL	7	10.99	2024-01-01
2	750mL	6	36.99	2024-01-01
3	750mL	3	38.99	2024-01-01
4	750mL	6	34.99	2024-01-01

----- end_inventory -----
Count of records: 224489

	InventoryId	Store	City	Brand	Description \
0	1_HARDERSFIELD_58	1	HARDERSFIELD	58	Gekkeikan Black & Gold Sake
1	1_HARDERSFIELD_62	1	HARDERSFIELD	62	Herradura Silver Tequila
2	1_HARDERSFIELD_63	1	HARDERSFIELD	63	Herradura Reposado Tequila
3	1_HARDERSFIELD_72	1	HARDERSFIELD	72	No. 3 London Dry Gin
4	1_HARDERSFIELD_75	1	HARDERSFIELD	75	Three Olives Tomato Vodka

	Size	onHand	Price	endDate
0	750mL	11	12.99	2024-12-31
1	750mL	7	36.99	2024-12-31
2	750mL	7	38.99	2024-12-31
3	750mL	4	34.99	2024-12-31
4	750mL	7	14.99	2024-12-31

----- sales -----
Count of records: 12825363

	InventoryId	Store	Brand	Description	Size \
0	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL
1	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL
2	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL
3	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL
4	1_HARDERSFIELD_1005	1	1005	Maker's Mark Combo Pack	375mL 2 Pk

	SalesQuantity	SalesDollars	SalesPrice	SalesDate	Volume \
0	1	16.49	16.49	2024-01-01	750.0
1	2	32.98	16.49	2024-01-02	750.0
2	1	16.49	16.49	2024-01-03	750.0
3	1	14.49	14.49	2024-01-08	750.0
4	2	69.98	34.99	2024-01-09	375.0

	Classification	ExciseTax	VendorNo	VendorName
0	1	0.79	12546	JIM BEAM BRANDS COMPANY
1	1	1.57	12546	JIM BEAM BRANDS COMPANY
2	1	0.79	12546	JIM BEAM BRANDS COMPANY
3	1	0.79	12546	JIM BEAM BRANDS COMPANY
4	1	0.79	12546	JIM BEAM BRANDS COMPANY

----- vendor_sales_summary -----
 --
 Count of records: 10692

	VendorNumber	VendorName	Brand	Description \
0	1128	BROWN-FORMAN CORP	1233	Jack Daniels No 7 Black
1	4425	MARTIGNETTI COMPANIES	3405	Tito's Handmade Vodka
2	17035	PERNOD RICARD USA	8068	Absolut 80 Proof
3	3960	DIAGEO NORTH AMERICA INC	4261	Capt Morgan Spiced Rum
4	3960	DIAGEO NORTH AMERICA INC	3545	Ketel One Vodka

	PurchasePrice	ActualPrice	Volume	TotalPurchaseQuantity	TotalPurchaseDollars	TotalSalesQuantity
TotalSalesDollars						
0	26.27	36.99	1750.0	145080	3811251.60	142049.0
5.101920e+06						
1	23.19	28.99	1750.0	164038	3804041.22	160247.0
4.819073e+06						
2	18.24	24.99	1750.0	187407	3418303.68	187140.0
4.538121e+06						
3	16.17	22.99	1750.0	201682	3261197.94	200412.0
4.475973e+06						
4	21.89	29.99	1750.0	138109	3023206.01	135838.0
4.223108e+06						

	TotalSalesPrice	TotalExciseTax	FreightCost	GrossProfit	ProfitMargin	StockTurnover
SalesToPurchaseRatio						
0	672819.31	260999.20	68601.68	1290667.91	25.297693	0.979108
1.338647						
1	561512.37	294438.66	144929.24	1015032.27	21.062810	0.976890
1.266830						
2	461140.15	343854.07	123780.22	1119816.92	24.675786	0.998575
1.327594						
3	420050.01	368242.80	257032.07	1214774.94	27.139908	0.993703
1.372493						
4	545778.28	249587.83	257032.07	1199901.61	28.412764	0.983556
1.396897						

- The begin_inventory and end_inventory tables contain inventory data at the start and end of the year, which is not relevant for analyzing vendor behavior.
- Therefore, these tables can be excluded from the analysis.
- Next, we will examine how vendor-related data is distributed across other tables.

```
# choosing one vendor and filtering the different tables
purchases = pd.read_sql_query("SELECT * FROM purchases WHERE VendorNumber = 4466",conn)
purchases
```

[2192 rows x 16 columns] – same record-level detail as the SQLite version, filtered to VendorNumber 4466 (AMERICAN VINTAGE BEVERAGE).

```
purchase_prices = pd.read_sql_query("""SELECT * FROM purchase_prices WHERE VendorNumber = 4466""",conn)
purchase_prices
```

	Brand	Description	Price	Size	Volume	Classification \
0	5215	TGI Fridays Long Island Iced	12.99	1750mL	1750	1
1	5255	TGI Fridays Ultimte Mudslide	12.99	1750mL	1750	1
2	3140	TGI Fridays Orange Dream	14.99	1750mL	1750	1

	PurchasePrice	VendorNumber	VendorName
0	9.41	4466	AMERICAN VINTAGE BEVERAGE
1	9.35	4466	AMERICAN VINTAGE BEVERAGE
2	11.19	4466	AMERICAN VINTAGE BEVERAGE

```
vendor_invoice = pd.read_sql_query("""SELECT * FROM vendor_invoice WHERE VendorNumber = 4466""",conn)
vendor_invoice
```

[55 rows x 8 columns] – PO-level invoice records for vendor 4466, identical content to the SQLite result (PONumbers 8137 through 13627, spanning 2023-12-22 to 2025-01-09).

```
sales = pd.read_sql_query("""SELECT * FROM sales WHERE VendorNo = 4466""",conn)
sales
```

[9453 rows x 14 columns] – transaction-level sales records for vendor 4466 (Brands 5215/5255/3140 – TGI Fridays cocktail line).

```
purchases.groupby(["Brand", "PurchasePrice"])[["Quantity", "Dollars"]].sum()
```

Brand	PurchasePrice	Quantity	Dollars
3140	11.19	4640	51921.60
5215	9.41	4923	46325.43
5255	9.35	6215	58110.25

```
purchases.groupby(["PONumber"])[["Quantity", "Dollars"]].sum()
```

[55 rows x 2 columns] – same per-PO totals as the SQLite version (PONumber 8137 → Quantity 15, Dollars 140.55 ... PONumber 13595 → Quantity 413, Dollars 3985.03)

```
sales.groupby("Brand")[['SalesQuantity', 'SalesDollars', 'SalesPrice', 'ExciseTax']].sum()
```

Brand	SalesQuantity	SalesDollars	SalesPrice	ExciseTax
3140	3890	50531.10	30071.85	7149.25
5215	4651	60416.49	41542.02	8548.96
5255	6096	79187.04	51180.60	11204.28

- The purchases table contains actual purchase data, including the date of purchase, products (brands) purchased by vendors, the amount paid (in dollars), and the quantity purchased.
- The purchase price column is derived from the purchase_prices table, which provides product-wise actual and purchase prices. The combination of vendor and brand is unique in this table.
- The vendor_invoice table aggregates data from the purchases table, summarizing quantity and dollar amounts, along with an additional column for freight. This table maintains uniqueness based on vendor and PO number.
- The sales table captures actual sales transactions, detailing the brands purchased by vendors, the quantity sold, the selling price, and the revenue earned.

As the data that we need for analysis is distributed in different tables, we need to create a summary table containing:

- purchase transactions made by vendors
- sales transaction data
- freight costs for each vendor

- actual product prices from vendors

Let's first explore each summary in different tables.

```
freight_summary = pd.read_sql_query("""SELECT
    VendorNumber,
    SUM(Freight) AS FreightCost
FROM vendor_invoice
GROUP BY VendorNumber""", conn)
freight_summary
```

	VendorNumber	FreightCost
0	2	27.08
1	54	0.48
2	60	367.52
3	105	62.39
4	200	6.19
..	...	...
121	98450	856.02
122	99166	130.09
123	172662	178.34
124	173357	202.50
125	201359	0.09

[126 rows x 2 columns]

```
purchase_summary = pd.read_sql_query("""SELECT
    p.VendorNumber,
    p.VendorName,
    p.Brand,
    p.Description,
    p.PurchasePrice,
    pp.Price AS ActualPrice,
    pp.Volume,
    SUM(p.Quantity) AS TotalPurchaseQuantity,
    SUM(p.Dollars) AS TotalPurchaseDollars
FROM purchases p
JOIN purchase_prices pp
    ON p.Brand = pp.Brand
WHERE p.PurchasePrice > 0
GROUP BY p.VendorNumber, p.VendorName, p.Brand, p.Description, p.PurchasePrice, pp.Price, pp.Volume
ORDER BY TotalPurchaseDollars""", conn)
purchase_summary
```

	VendorNumber	VendorName	Brand	\
0	7245	PROXIMO SPIRITS INC.	3065	
1	3960	DIAGEO NORTH AMERICA INC	6127	
2	3924	HEAVEN HILL DISTILLERIES	9123	
3	8004	SAZERAC CO INC	5683	
4	9815	WINE GROUP INC	8527	
...	...	...	...	
10687	3960	DIAGEO NORTH AMERICA INC	3545	
10688	3960	DIAGEO NORTH AMERICA INC	4261	
10689	17035	PERNOD RICARD USA	8068	
10690	4425	MARTIGNETTI COMPANIES	3405	
10691	1128	BROWN-FORMAN CORP	1233	

[10692 rows x 9 columns]

```
sales_summary = pd.read_sql_query("""SELECT
    VendorNo,
    Brand,
    SUM(SalesQuantity) AS TotalSalesQuantity,
    SUM(SalesDollars) AS TotalSalesDollars,
    SUM(SalesPrice) AS TotalSalesPrice,
    SUM(ExciseTax) AS TotalExciseTax
FROM sales
GROUP BY VendorNo, Brand""", conn)
sales_summary
```

[11272 rows x 6 columns] – per vendor/brand totals (VendorNo 2, Brand 90085 → TotalSalesQuantity 18 ... VendorNo 201359, Brand 90609 → TotalSalesQuantity 59)

```

# merging all the summary table to create an aggregated summary table for further analysis
vendor_sales_summary = pd.read_sql_query("""WITH FreightSummary AS (
    SELECT
        VendorNumber,
        SUM(Freight) AS FreightCost
    FROM vendor_invoice
    GROUP BY VendorNumber
),

PurchaseSummary AS (
    SELECT
        p.VendorNumber,
        p.VendorName,
        p.Brand,
        p.Description,
        p.PurchasePrice,
        pp.Price AS ActualPrice,
        pp.Volume,
        SUM(p.Quantity) AS TotalPurchaseQuantity,
        SUM(p.Dollars) AS TotalPurchaseDollars
    FROM purchases p
    JOIN purchase_prices pp
        ON p.Brand = pp.Brand
    WHERE p.PurchasePrice > 0
    GROUP BY p.VendorNumber, p.VendorName, p.Brand, p.Description, p.PurchasePrice, pp.Price, pp.Volume
),

SalesSummary AS (
    SELECT
        VendorNo,
        Brand,
        SUM(SalesQuantity) AS TotalSalesQuantity,
        SUM(SalesDollars) AS TotalSalesDollars,
        SUM(SalesPrice) AS TotalSalesPrice,
        SUM(ExciseTax) AS TotalExciseTax
    FROM sales
    GROUP BY VendorNo, Brand
)

SELECT
    ps.VendorNumber,
    ps.VendorName,
    ps.Brand,
    ps.Description,
    ps.PurchasePrice,
    ps.ActualPrice,
    ps.Volume,
    ps.TotalPurchaseQuantity,
    ps.TotalPurchaseDollars,
    ss.TotalSalesQuantity,
    ss.TotalSalesDollars,
    ss.TotalSalesPrice,
    ss.TotalExciseTax,
    fs.FreightCost
FROM PurchaseSummary ps
LEFT JOIN SalesSummary ss
    ON ps.VendorNumber = ss.VendorNo
    AND ps.Brand = ss.Brand
LEFT JOIN FreightSummary fs
    ON ps.VendorNumber = fs.VendorNumber
ORDER BY ps.TotalPurchaseDollars DESC""",conn)

vendor_sales_summary.columns

```

```

Index(['VendorNumber', 'VendorName', 'Brand', 'Description', 'PurchasePrice',
      'ActualPrice', 'Volume', 'TotalPurchaseQuantity',
      'TotalPurchaseDollars', 'TotalSalesQuantity', 'TotalSalesDollars',
      'TotalSalesPrice', 'TotalExciseTax', 'FreightCost'],

```

```
dtype='object')
```

This query generates a vendor-wise sales and purchase summary, which is valuable for:

Performance Optimization:

- The query involves heavy joins and aggregations on large datasets like sales and purchases.
- Storing the pre-aggregated results avoids repeated expensive computations.
- Helps in analyzing sales, purchases, and pricing for different vendors and brands.
- Future Benefits of Storing this data for faster Dashboarding & Reporting.
- Instead of running expensive queries each time, dashboards can fetch data quickly from vendor_sales_summary.

Let's clean the data if there is any inconsistencies.

Data Cleaning

```
vendor_sales_summary.dtypes
```

```
VendorNumber      int64
VendorName         object
Brand             int64
Description        object
PurchasePrice     float64
ActualPrice       float64
Volume            object
TotalPurchaseQuantity  int64
TotalPurchaseDollars  float64
TotalSalesQuantity  float64
TotalSalesDollars    float64
TotalSalesPrice     float64
TotalExciseTax      float64
FreightCost        float64
dtype: object
```

```
vendor_sales_summary.isnull().sum()
```

```
VendorNumber      0
VendorName         0
Brand             0
Description        0
PurchasePrice     0
ActualPrice       0
Volume            0
TotalPurchaseQuantity  0
TotalPurchaseDollars  0
TotalSalesQuantity 178
TotalSalesDollars  178
TotalSalesPrice   178
TotalExciseTax    178
FreightCost       0
dtype: int64
```

```
vendor_sales_summary['Volume'].unique()
```

```
array(['1750', '750', '1000', '1500', '375', '50', '3000', '5000', '100',
       '200', '4000', '187', '500', '600', '300', '1100', '250', '400',
       '18000', '150', '720', '330', '162.5', '180', '19500', '6000',
       '560', '20000', '9000', '3750', '650'], dtype=object)
```

```
vendor_sales_summary['VendorName'].unique()
```

```
array(['BROWN-FORMAN CORP', 'MARTIGNETTI COMPANIES', ... ,
       'FLAVOR ESSENCE INC'], dtype=object)
(126 distinct vendor names – identical to the source data, unchanged by the database engine)
```


- The Volume is a numerical column but having object datatype.
- There are some products that are not sold, so having missing values.
- There are white spaces in the categorical columns.

```
# creating new columns for better analysis
vendor_sales_summary['GrossProfit'] = vendor_sales_summary['TotalSalesDollars'] -
vendor_sales_summary['TotalPurchaseDollars']
vendor_sales_summary['ProfitMargin'] = (vendor_sales_summary['GrossProfit'] /
vendor_sales_summary['TotalSalesDollars'])*100
vendor_sales_summary['StockTurnover'] = vendor_sales_summary['TotalSalesQuantity'] /
vendor_sales_summary['TotalPurchaseQuantity']
vendor_sales_summary['SalesToPurchaseRatio'] = vendor_sales_summary['TotalSalesDollars'] /
vendor_sales_summary['TotalPurchaseDollars']

# changing datatype to float
vendor_sales_summary['Volume'] = vendor_sales_summary['Volume'].astype('float')

# filling missing value with 0
vendor_sales_summary.fillna(0,inplace = True)

# removing spaces from categorical columns
vendor_sales_summary['VendorName'] = vendor_sales_summary['VendorName'].str.strip()
vendor_sales_summary['Description'] = vendor_sales_summary['Description'].str.strip()
```

Saving cleaned data into database

```
cursor = conn.cursor()

# only runs once
cursor.execute("""CREATE TABLE vendor_sales_summary (
    VendorNumber INT,
    VendorName VARCHAR(100),
    Brand INT,
    Description VARCHAR(100),
    PurchasePrice DECIMAL(10,2),
    ActualPrice DECIMAL(10,2),
    Volume INT,
    TotalPurchaseQuantity INT,
    TotalPurchaseDollars DECIMAL(15,2),
    TotalSalesQuantity INT,
    TotalSalesDollars DECIMAL(15,2),
    TotalSalesPrice DECIMAL(15,2),
    TotalExciseTax DECIMAL(15,2),
    FreightCost DECIMAL(15,2),
    GrossProfit DECIMAL(15,2),
    ProfitMargin DECIMAL(15,2),
    StockTurnover DECIMAL(15,2),
    SalesToPurchaseRatio DECIMAL(15,2),
    PRIMARY KEY (VendorNumber, Brand)
) ENGINE=InnoDB;""")
conn.commit()
```

```
from sqlalchemy import create_engine

# SQLAlchemy engine is required for to_sql() with MySQL
engine = create_engine("mysql+pymysql://root:your_password@localhost/inventory")

vendor_sales_summary.to_sql('vendor_sales_summary', engine, if_exists='replace', index=False)
```

10692

```
pd.read_sql_query("SELECT * FROM vendor_sales_summary",conn)
```

VendorNumber	VendorName	Brand	\
--------------	------------	-------	---

0	1128	BROWN-FORMAN CORP	1233
1	4425	MARTIGNETTI COMPANIES	3405
2	17035	PERNOD RICARD USA	8068
3	3960	DIAGEO NORTH AMERICA INC	4261
4	3960	DIAGEO NORTH AMERICA INC	3545
...	...	...	...
10687	9815	WINE GROUP INC	8527
10688	8004	SAZERAC CO INC	5683
10689	3924	HEAVEN HILL DISTILLERIES	9123
10690	3960	DIAGEO NORTH AMERICA INC	6127
10691	7245	PROXIMO SPIRITS INC.	3065

	Description	PurchasePrice	ActualPrice	Volume \
0	Jack Daniels No 7 Black	26.27	36.99	1750.0
1	Tito's Handmade Vodka	23.19	28.99	1750.0
2	Absolut 80 Proof	18.24	24.99	1750.0
3	Capt Morgan Spiced Rum	16.17	22.99	1750.0
4	Ketel One Vodka	21.89	29.99	1750.0
...	...	...	...	...
10687	Concannon Glen Ellen Wh Zin	1.32	4.99	750.0
10688	Dr McGillicuddy's Apple Pie	0.39	0.49	50.0
10689	Deep Eddy Vodka	0.74	0.99	50.0
10690	The Club Strawbry Margarita	1.47	1.99	200.0
10691	Three Olives Grape Vodka	0.71	0.99	50.0

	TotalPurchaseQuantity	TotalPurchaseDollars	TotalSalesQuantity \
0	145080	3811251.60	142049.0
1	164038	3804041.22	160247.0
2	187407	3418303.68	187140.0
3	201682	3261197.94	200412.0
4	138109	3023206.01	135838.0
...	...	...	...
10687	2	2.64	5.0
10688	6	2.34	134.0
10689	2	1.48	2.0
10690	1	1.47	72.0
10691	1	0.71	86.0

	TotalSalesDollars	TotalSalesPrice	TotalExciseTax	FreightCost \
0	5.101920e+06	672819.31	260999.20	68601.68
1	4.819073e+06	561512.37	294438.66	144929.24
2	4.538121e+06	461140.15	343854.07	123780.22
3	4.475973e+06	420050.01	368242.80	257032.07
4	4.223108e+06	545778.28	249587.83	257032.07
...	...	...	...	...
10687	1.595000e+01	10.96	0.55	27100.41
10688	6.566000e+01	1.47	7.04	50293.62
10689	1.980000e+00	0.99	0.10	14069.87
10690	1.432800e+02	77.61	15.12	257032.07
10691	8.514000e+01	33.66	4.46	38994.78

	GrossProfit	ProfitMargin	StockTurnover	SalesToPurchaseRatio
0	1290667.91	25.297693	0.979108	1.338647
1	1015032.27	21.062810	0.976890	1.266830
2	1119816.92	24.675786	0.998575	1.327594
3	1214774.94	27.139908	0.993703	1.372493
4	1199901.61	28.412764	0.983556	1.396897
...	...	...	...	...
10687	13.31	83.448276	2.500000	6.041667
10688	63.32	96.436186	22.333333	28.059829
10689	0.50	25.252525	1.000000	1.337838
10690	141.81	98.974037	72.000000	97.469388
10691	84.43	99.166079	86.000000	119.915493

[10692 rows x 18 columns]